GASADVANCE AI

Advanced Backend Solution for Smart Gas Management & Predictive Analytics

PROJECT OVERVIEW

GasAdvance AI is a high-performance backend infrastructure designed to automate gas supply chains. Built with .NET 8 and Clean Architecture, the system integrates AI models to provide real-time forecasting, leak detection, and automated inventory management, ensuring safety and operational efficiency.

KEY CAPABILITIES

Predictive Analytics: Leverages machine learning to forecast consumption patterns and optimize distribution routes.

IoT Data Ingestion: Real-time processing of sensor data for pressure monitoring and immediate hazard alerting.

Enterprise Security: Secure API access via JWT Bearer tokens and comprehensive Role Based Access Control (RBAC).

TECHNOLOGICAL FOUNDATION

.NET 8 Core | ASP.NET Web API | Clean Architecture | SQL Server | Entity Framework Core | Swagger/OpenAPI 3.0 | JWT Auth

API SPECIFICATION (SWAGGER UI OVERVIEW)

POST	/api/v1/auth/identity	// Authentication & JWT Generation
GET	/api/v1/forecast/demand	// AI-Driven Supply Forecasting
POST	/api/v1/telemetry/sensors	// Ingesting Real-Time Sensor Metrics
GET	/api/v1/storage/levels	// Real-time Inventory Management
PUT	/api/v1/thresholds/safety	// Updating AI Alert Parameters

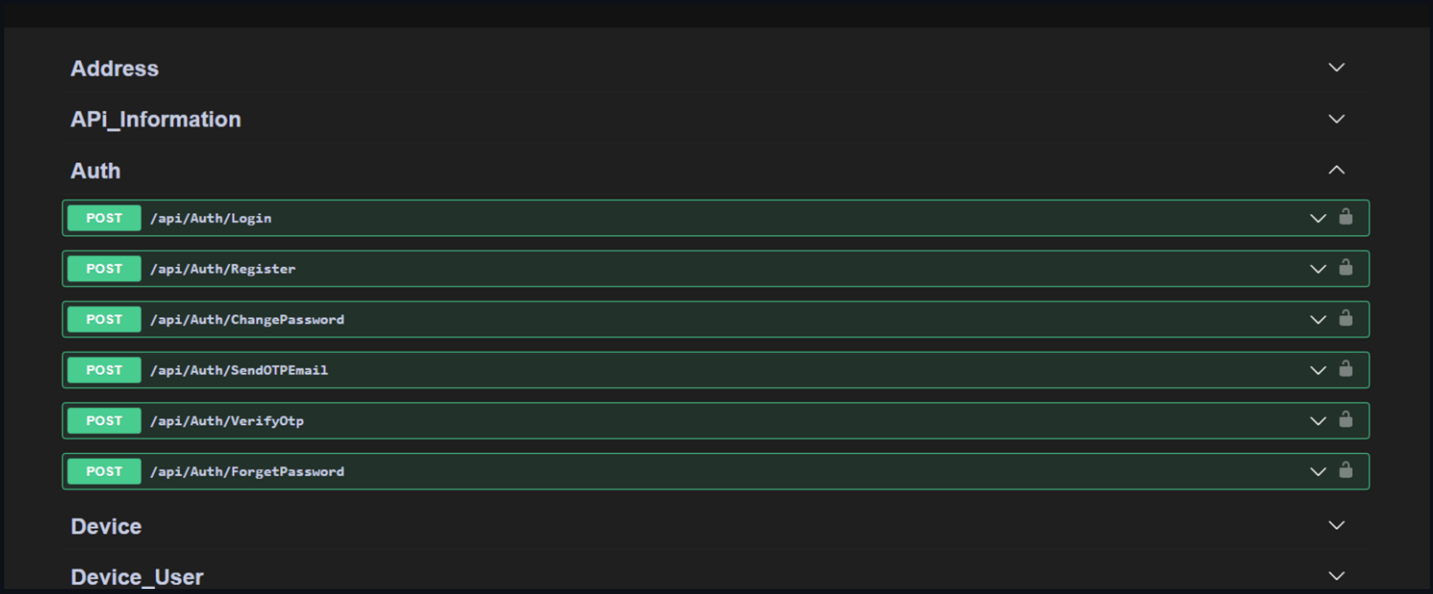
SOURCE CODE IMPLEMENTATION

```
[Route("api/[controller]")]
[ApiController]
1 reference
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;
    0 references
    public AuthController(IAuthService authenticationService)
    {
        _authService = authenticationService;

    }
    [HttpPost("Login")]
    0 references
    public async Task<GeneralResponse<AuthResponse>> Login(AuthRequest request)
    {

       return await _authService.Login(request);
    }
    [HttpPost("Register")]
    0 references
    public async Task<GeneralResponse<RegistrationResponse>> Register(RegistrationRequest request)
    {

       return await _authService.Register(request);
    }
    [Authorize]
    [HttpPost("ChangePassword")]
    0 references
    public async Task<GeneralResponse<ChangeUserPasswordResponse>> ChangePassword(ChangeUserPasswordRequest request)
    {
        return await _authService.ChangePassword(request);
    }
    [HttpPost("SendOTPEmail")]
    0 references
    public async Task<GeneralResponse<ForgetPasswordResponse>> SendOTPEmail(string email)
    {
        return await _authService.SendOTP(email);
    }

    [HttpPost("VerifyOtp")]
    0 references
    public async Task<GeneralResponse<AuthResponse>> VerifyOtp(VerifyOtpRequest request)
    {
        return await _authService.VerifyOtp(request);
    }
```

```
2 references
public async Task<GeneralResponse<AuthResponse>> Login(AuthRequest request)
{
    CheckIsExist

        if (string.IsNullOrEmpty(request.DeviceToken))
        {
            return new GeneralResponse<AuthResponse>(_localization["Device token is required."].Value, 0);

        }
        else
        {

            // Register or update the device token for the logged-in user
            var existingDevice = _unit.UserApp.All().Where(d => d.UserId == User.Id && d.AppToken == request.DeviceToken).FirstOrDefault();

            if (existingDevice == null)
            {
                UserAppInput userApp = new UserAppInput
                {
                    UserId = User.Id,
                    AppToken = request.DeviceToken
                };
                await _UserAppService.Add(userApp, User.Id);

            }
        }



    string ImagePath = "", FileName = "";
    Stuf
    User
    Generate JWT
    return new GeneralResponse<AuthResponse>(response, _localization["LoginSuccessfully"].Value, 0);

}
```

Service Layer and Logic implementation

Clean Code & Architecture Standards

The implementation follows the **Separation of Concerns** principle, ensuring that business logic is decoupled from external dependencies. By utilizing the **Repository Pattern** and **Dependency Injection**, the codebase remains highly maintainable, testable, and scalable for future enterprise requirements.

INTERACTIVE DOCUMENTATION (SWAGGER)